

Full Recommended Stack

Project 1 — Spatiotemporal Graph Engine (ML/Data Backend)

Category	Choice	Why
Language	Python 3.11+	Ecosystem dominance for all sports analytics and ML libraries
Data ingestion	statsbombpy , kloppy	Official parsers for StatsBomb and SkillCorner — Gemini confirmed these
Action valuation	socceraction	SPADL conversion, xT and VAEP calculation out of the box
Graph construction	NetworkX	Best for passing network construction, centrality metrics, TVG building
GNN / deep graph ML	PyTorch Geometric	Standard for spatiotemporal GNNs, role permutation modeling
Hungarian algorithm	scipy.optimize.linear_sum_assignment	Built-in bipartite matching, no extra dependency needed
CRF / Viterbi	torchcrf or sklearn-crfsuite	For physically constrained state parsing
Data processing	pandas , numpy , polars	Polars for speed on large tracking datasets, pandas for event data
Spatial geometry	shapely	Convex hulls, Delaunay triangulation, Voronoi cells
Synchronization	databallpy	Event-tracking timeline alignment
Storage	PostgreSQL	Structured storage for matches, events, computed graph states, metrics

Category	Choice	Why
Caching	Redis	Pre-computed spatial topologies cached at 5Hz downsampled rate
API layer	FastAPI	Async, fast, typed — serves computed metrics to frontend and agent
Task queue	Celery + Redis	Async processing for heavy graph computation jobs
Experiment tracking	MLflow	Track model runs, ablation experiments, baselines for paper
Testing	pytest	Module-level unit tests for pipeline integrity

Project 2 — Dynamic Passing & Formation Visualizer (Frontend)

This is the FC26-style tactical board layer. The goal is **interactive, animated, real-time looking** — not matplotlib plots.

Category	Choice	Why
Frontend framework	React + TypeScript	Industry standard, component-based, clean separation of concerns
Pitch rendering	<code>mplsoccer</code> (backend export) + <code>D3.js</code> (frontend)	D3 for fully custom animated SVG pitch views, passing networks, and flowing arrows
3D / animated tactics	<code>Three.js</code> or <code>Deck.gl</code>	Deck.gl for GPU-accelerated moving player positions and Voronoi overlays
Tactical board UI	<code>Konva.js</code> or <code>Fabric.js</code>	Canvas-based interactive tactical board — drag players, draw formations
Charts and tables	<code>Recharts</code> or <code>Nivo</code>	Clean React-native charts for stats, player metrics, network graphs
Graph visualization	<code>Cytoscape.js</code>	Interactive passing network graphs with edge weights, centrality coloring

Category	Choice	Why
Animation	Framer Motion	Smooth tactical phase transitions and player movement animations
State management	Zustand	Lightweight, simple state for match phase selection, player filters
API calls	Axios + React Query	Efficient data fetching from your FastAPI backend
Styling	Tailwind CSS	Fast, clean UI without writing custom CSS from scratch
Build tool	Vite	Fast dev server and build for React

What this produces visually:

- Animated passing network that shifts as match phases change
- Moving Voronoi cells showing spatial control
- Formation overlay that morphs between detected tactical shapes
- Player cards with xT/VAEP scores, centrality rankings
- Timeline scrubber to move through match phases
- Convex hull showing team compactness over time

Project 3 — Agentic Match Analyst (Orchestration Layer)

Category	Choice	Why
LLM provider	Groq Cloud API	Your choice — extremely fast inference, supports LLaMA 3.x and Mixtral
Recommended model	LLaMA 3.3 70B on Groq	Best balance of reasoning quality and speed on Groq
Agent framework	LangGraph	Better than plain LangChain for multi-agent workflows — supports stateful graph-based agent orchestration
Tool calling	LangGraph tool nodes	Each specialized agent gets its own deterministic tools
Vector DB	ChromaDB	Lightweight, local-first, easy to deploy — stores semantic match summaries

Category	Choice	Why
Embeddings	sentence-transformers	Local embedding generation for tactical report retrieval
Report generation	Custom Report Synthesizer agent	LLM-as-a-Judge pattern confirmed by Gemini
Memory / session	LangGraph checkpointing + PostgreSQL	Persistent conversation history per match session
Agent UI	React frontend (same as Project 2)	Chat interface embedded in the same dashboard
Evaluation	RAGAS	Measures retrieval quality, factuality, MRR for the agentic layer

Five agents confirmed by Gemini:

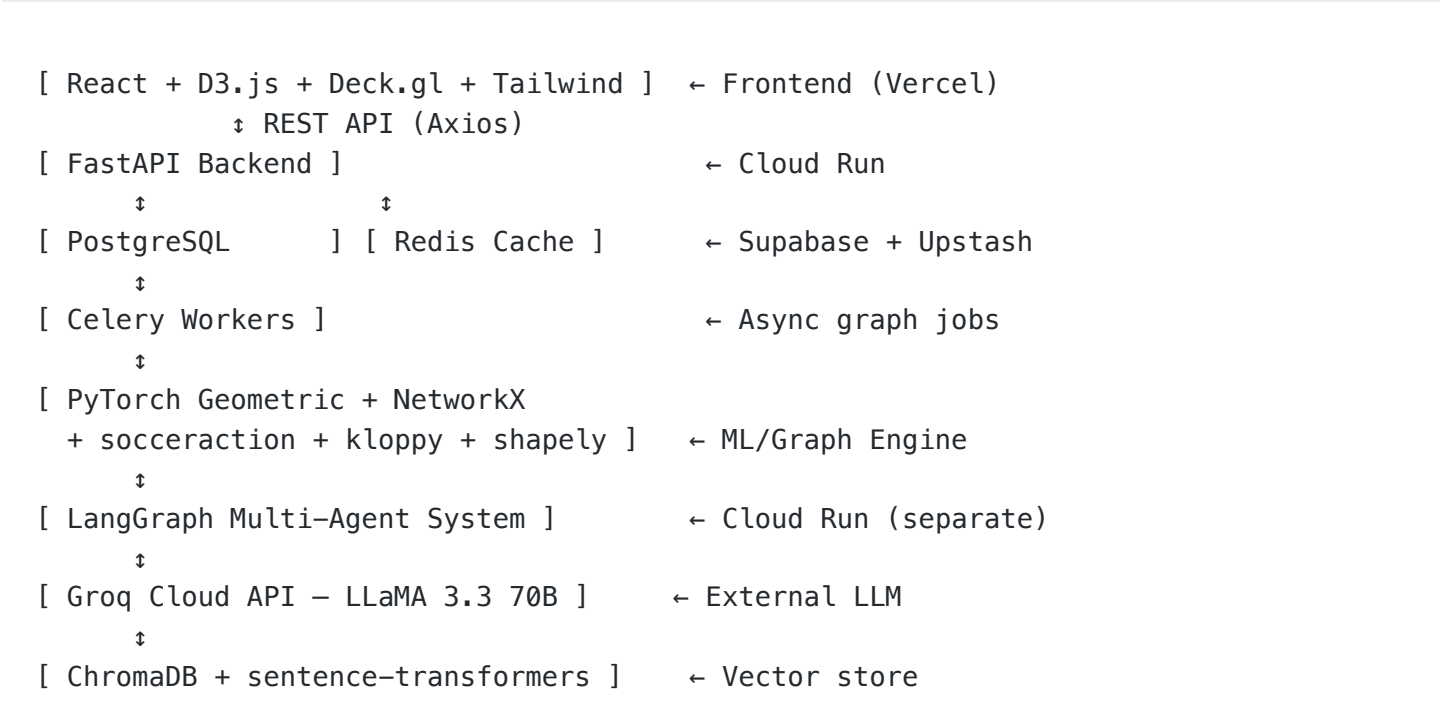
1. Orchestrator Agent — intent parsing, session management
2. Spatial Topology Agent — centroids, convex hulls, Delaunay, role permutations
3. Network Analytics Agent — TVG centrality, clustering coefficients
4. Valuation Engine Agent — xT/VAEP queries from PostgreSQL
5. Report Synthesizer / LLM-as-a-Judge — drafts and validates final report

Cross-cutting — Database, DevOps, Deployment

Category	Choice	Why
Primary DB	PostgreSQL on Supabase	You already use Supabase — managed, cloud-native, free tier sufficient
Cache	Redis on Railway or Upstash	Upstash is serverless Redis, cheapest for cloud
Backend deployment	Google Cloud Run	You already use it — containerized FastAPI, scales to zero
Frontend deployment	Vercel	Best for React — instant deploys, free tier, CDN included
Agent service	Google Cloud Run (separate container)	Isolated from backend, scales independently

Category	Choice	Why
Containerization	Docker + Docker Compose	One compose file runs everything locally, same image goes to Cloud Run
CI/CD	GitHub Actions	Auto-deploy on push, run tests, build containers
Environment management	python-dotenv + .env files	Groq API key, Supabase keys, all managed cleanly
Monitoring	Grafana + Prometheus or just Cloud Run logs	Start with Cloud Run logs, add Grafana later for paper metrics

Full Stack Visual Summary



Three things to note

1. **Your M4 MacBook Air handles this entire stack locally** via Docker Compose for development — no GPU needed for v1 since PyTorch Geometric graph models at your scale are CPU-feasible, and Groq handles LLM inference externally.
2. **LangGraph over plain LangChain** is the right call for your 5-agent architecture because it models agent workflows as stateful graphs, which maps perfectly to your Orchestrator → Specialized Agents → Synthesizer pattern.

3. **D3.js + Deck.gl for the frontend** is what gives you the FC26-style moving visualizations — Deck.gl is GPU-accelerated in browser and can handle animated player positions smoothly, while D3 handles the passing network graph animations and tactical overlays.